

Memoria del Proyecto: API REST para Agricultura Inteligente

Grupo 9

JUAN DEL POZO ÁVILA
JUAN LOPEZ SUAREZ
MIGUEL PINTO VELASCO

16 de mayo de 2026

Resumen

El presente documento describe el diseño, la arquitectura y la implementación de una API REST orientada al ámbito de la agricultura inteligente. Este sistema actúa como un nodo sumidero en una red de Internet de las Cosas (IoT), recibiendo datos de diferentes motas o dispositivos distribuidos en diversas parcelas agrícolas. Las métricas recolectadas se procesan, almacenan y exponen haciendo uso de tecnologías web modernas mediante el framework Spring Boot. Además, se detalla el uso del estándar SenML (Sensor Measurement Lists, RFC 8428) para la estructuración unificada de los datos telemétricos, así como la aplicación de los conceptos teóricos adquiridos durante las diferentes fases de aprendizaje práctico (Prácticas 1 y 2).

Índice

1	Introducción	3
1.1	Contexto del Proyecto	3
1.2	Uso del estándar SenML (RFC 8428)	3
2	Aplicación de Contenidos Prácticos	4
2.1	Contenidos de la Práctica 1: Fundamentos REST	4
2.1.1	Anotaciones Principales: @RestController y @SpringBootApplication	4
2.1.2	Peticiones GET y uso de @RequestParam	4
2.2	Contenidos de la Práctica 2: REST Avanzado	5
2.2.1	Mappings para todos los métodos HTTP	5
2.2.2	Variables de Ruta: @PathVariable	5
2.2.3	Manejo del cuerpo de petición: @RequestBody	5
2.2.4	Control exhaustivo con ResponseEntity	5
2.2.5	Documentación Automática: Swagger/OpenAPI	5
3	Arquitectura y Patrones de Diseño	7
3.1	Patrón MVC (Model-View-Controller) / Capa de Controladores	7
3.2	Patrón DAO (Data Access Object)	7
4	Diseño del Sistema (Diagramas UML)	8
4.1	Diagrama de Clases	8
4.2	Diagramas de Secuencia	9
4.2.1	Operación PUT de Parcelas	9
4.2.2	Operación POST de Medidas (SenML)	9
4.2.3	Operación DELETE de un Dispositivo	10
5	Conclusiones	11

1. Introducción

1.1. Contexto del Proyecto

La agricultura de precisión e inteligente es un campo en constante evolución, impulsado por el abaratamiento de los sensores y la ubicuidad de las redes inalámbricas de área personal (WPAN). En este entorno, los cultivos se monitorizan mediante redes de sensores inalámbricos. Cada campo de cultivo, denominado *parcela*, dispone de múltiples nodos sensores o *motas* (denominados en este proyecto *dispositivos*).

Estos dispositivos capturan variables físicas como la humedad del suelo, la temperatura ambiente, la radiación solar y la concentración de fertilizantes. Debido a las limitaciones energéticas de las motas, estas transmiten la información cruda a un nodo más potente, conocido como nodo sumidero. El objetivo de este proyecto ha sido implementar el software residente en dicho nodo sumidero: una API RESTful capaz de recibir, procesar y almacenar estas mediciones, y posteriormente ofrecerlas de manera estructurada a clientes de la red global.

1.2. Uso del estándar SenML (RFC 8428)

Para dotar de interoperabilidad al sistema, se ha adoptado el estándar SenML (*Sensor Measurement Lists*), definido en el RFC 8428 del IETF. SenML propone un modelo de datos simple y eficiente para representar medidas de sensores. La elección de SenML se justifica por su capacidad de agrupar múltiples mediciones en un solo mensaje JSON (o CBOR, XML, etc.), reduciendo el *overhead* de las cabeceras HTTP.

En el proyecto se ha utilizado la librería externa *SenML API* alojada en GitHub. Este componente facilita el parseo de mensajes JSON crudos a objetos Java manejables por la aplicación, permitiendo extraer atributos fundamentales como el *Base Name* (bn), el *Base Time* (bt), el nombre de la variable (n), su unidad (u) y el valor en sí (v, vb, o vs dependiendo de si es numérico, booleano o cadena).

2. Aplicación de Contenidos Prácticos

Durante el desarrollo del proyecto se han aplicado progresivamente los conceptos abordados en las prácticas de la asignatura. A continuación, se detallan y analizan estas aplicaciones teóricas y prácticas.

2.1. Contenidos de la Práctica 1: Fundamentos REST

En la primera fase de desarrollo (P1), se establecieron los cimientos de la aplicación web utilizando Spring Boot, un framework que abstrae gran parte de la configuración base de Spring.

2.1.1. Anotaciones Principales: @RestController y @SpringBootApplication

El punto de entrada del sistema está anotado con @SpringBootApplication. Esta etiqueta realiza de manera automática la configuración de los beans, el escaneo de componentes (*component scanning*) dentro del paquete base, y activa las auto-configuraciones propias de Spring Boot.

Para definir la exposición web, se ha utilizado la anotación @RestController en las clases ParcelaController, DispositivoController y MedicionController. A diferencia del clásico @Controller, la etiqueta @RestController asume la semántica de @ResponseBody por defecto. Esto significa que cualquier objeto devuelto por los métodos de estas clases será serializado automáticamente a JSON e inyectado en el cuerpo de la respuesta HTTP, lo cual es el comportamiento idóneo para una API.

2.1.2. Peticiones GET y uso de @RequestParam

Se implementaron inicialmente operaciones de solo lectura para listar los recursos disponibles. Para ello, se empleó la anotación @GetMapping. Uno de los requisitos implementados ha sido el uso de la etiqueta @RequestParam para la obtención de parámetros a través de la *query string* de la URI (lo que comúnmente aparece tras el símbolo de interrogación ?).

```
1 @GetMapping(value = "/parcelas", params = "id")
2 public ResponseEntity<Parcela> getParcela(@RequestParam(value = "id")
   int id) {
3     Parcela parcela = dao.getId(id);
4     if (parcela != null) {
5         return new ResponseEntity<>(parcela, HttpStatus.OK);
6     } else {
7         return new ResponseEntity<>(HttpStatus.NOT_FOUND);
8     }
9 }
```

Listing 1: Ejemplo de uso de RequestParam

Como se observa, mediante ?id=X, el cliente especifica la parcela deseada. Para evitar ambigüedades de ruteo con el método que devuelve todas las parcelas (mapeado también en /parcelas), se añadió el modificador params = id" dentro de @GetMapping.

2.2. Contenidos de la Práctica 2: REST Avanzado

La segunda fase amplió el sistema hasta convertirlo en un servicio CRUD (Create, Read, Update, Delete) completo, introduciendo operaciones más complejas y herramientas de documentación.

2.2.1. Mappings para todos los métodos HTTP

Se completó el conjunto de verbos HTTP utilizando sus respectivas anotaciones en Spring:

- **@PostMapping:** Utilizado para crear nuevos recursos (parcelas, dispositivos) y para el envío de nuevas mediciones (SenML).
- **@PutMapping:** Empleado para la actualización completa de una parcela o dispositivo existente.
- **@DeleteMapping:** Usado para eliminar recursos cuando una mota es retirada o una parcela es dada de baja.

2.2.2. Variables de Ruta: @PathVariable

A diferencia de **@RequestParam**, que se utiliza preferentemente para filtrado y paginación, la convención REST dicta que los identificadores de recursos jerárquicos deben formar parte del *path* de la URI. Para gestionar las mediciones vinculadas a una entidad concreta, se utilizó **@PathVariable**.

En la clase **MedicionController**, esta etiqueta es esencial, dado el diseño de las URIs: `/parcelas/{parcelaId}/mediciones/senml`.

2.2.3. Manejo del cuerpo de petición: @RequestBody

Cuando un cliente desea registrar un nuevo recurso, envía la representación en formato JSON en el cuerpo (*body*) de la petición HTTP POST o PUT. La anotación **@RequestBody** indica a Spring que debe interceptar dicho JSON y, haciendo uso de la librería Jackson, deserializarlo automáticamente en un objeto Java de la clase correspondiente (por ejemplo, una clase **Parcela**). Se utilizan etiquetas como **@JsonProperty** en los DTOs si el nombre del campo JSON difiere del nombre del atributo en Java.

2.2.4. Control exhaustivo con ResponseEntity

En una API profesional, no basta con devolver datos; el servidor debe emitir códigos de estado HTTP semánticamente correctos. Se ha hecho un uso sistemático de la clase genérica **ResponseEntity<T>**. Esto ha permitido que el servidor responda con 201 **CREATED** al crear recursos, 404 **NOT FOUND** si se solicita un identificador inexistente, y 200 **OK** para operaciones exitosas regulares.

2.2.5. Documentación Automática: Swagger/OpenAPI

Para asegurar que los consumidores de la API entiendan su contrato, se integró Swagger/OpenAPI mediante dependencias de SpringDoc. Esta herramienta inspecciona las anotaciones mencionadas previamente y expone una interfaz visual (normalmente en

`/swagger-ui.html`) donde se documentan todos los *endpoints*, modelos de datos y esquemas de petición y respuesta, posibilitando pruebas en vivo sin necesidad de un cliente intermedio.

3. Arquitectura y Patrones de Diseño

El sistema ha sido modelado para ser mantenible y desacoplado, aplicando patrones de arquitectura de software ampliamente aceptados.

3.1. Patrón MVC (Model-View-Controller) / Capa de Controladores

Aunque al ser una API REST no existe una *Vista* gráfica en el sentido tradicional (HTML/CSS renderizado en servidor), el patrón MVC se adapta, donde la vista es esencialmente la serialización de datos a JSON y el cliente REST. Los controladores (`ParcelaController`, `DispositivoController`, `MedicionController`) actúan como la capa HTTP. Su única responsabilidad es recibir peticiones web, validar el formato básico, delegar la lógica de negocio a las capas inferiores, y transformar los resultados en respuestas HTTP conformes.

3.2. Patrón DAO (Data Access Object)

Para desacoplar el acceso a datos del comportamiento web, se implementó el patrón DAO. Las clases `ParcelaDAO`, `DispositivoDAO` y `MedicionDAO` encapsulan la lógica para insertar, recuperar, actualizar y eliminar entidades de la base de datos (u origen de datos en memoria). El controlador no posee conocimiento de si la persistencia es una base de datos relacional PostgreSQL, un archivo plano o una base de datos NoSQL. Si en el futuro la persistencia cambia, los controladores no requerirán de modificación alguna.

4. Diseño del Sistema (Diagramas UML)

A continuación, se exponen los diagramas que modelan el comportamiento estático y dinámico de la aplicación.

4.1. Diagrama de Clases

El siguiente diagrama detalla la relación estructural entre los modelos, los DAOs y los Controladores del dominio.

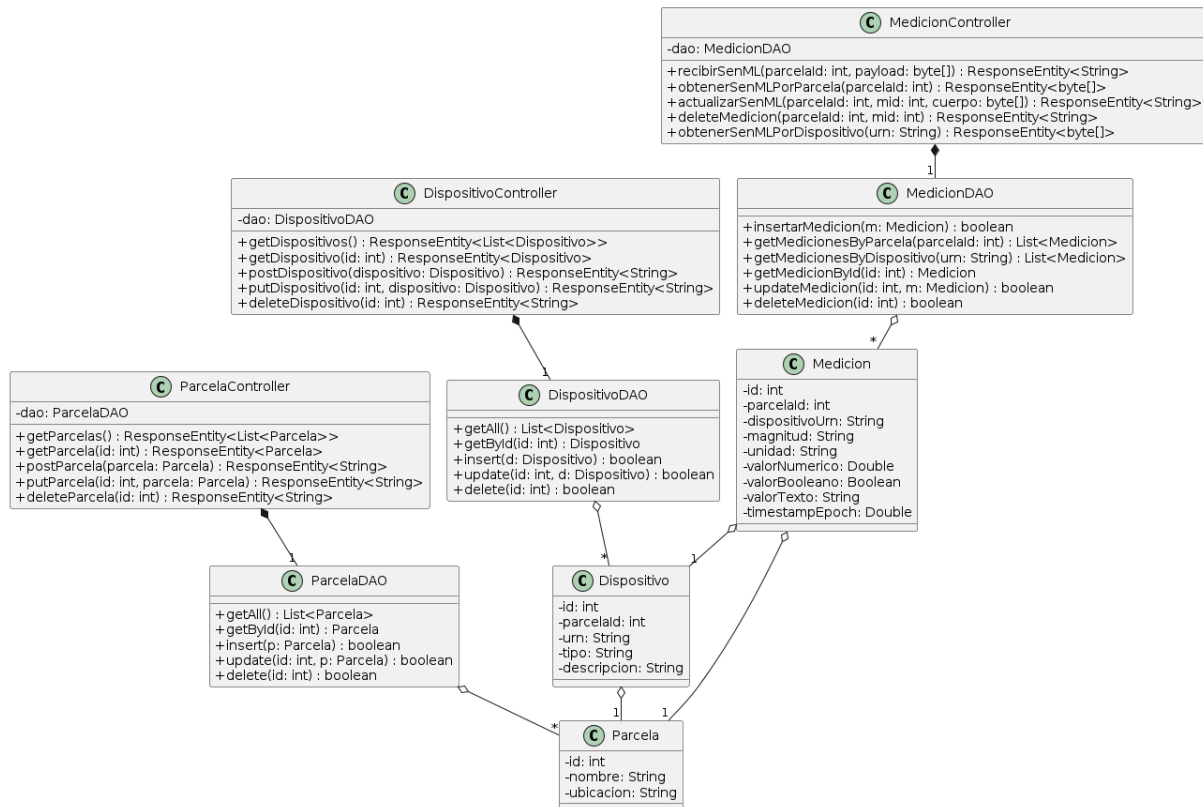


Figura 1: Diagrama de Clases

4.2. Diagramas de Secuencia

Para comprender la ejecución dinámica de las operaciones clave de la API, se exponen tres diagramas de secuencia fundamentales.

4.2.1. Operación PUT de Parcelas

Este diagrama muestra el flujo cuando un cliente solicita actualizar los datos de una parcela existente utilizando un parámetro de consulta para el ID.

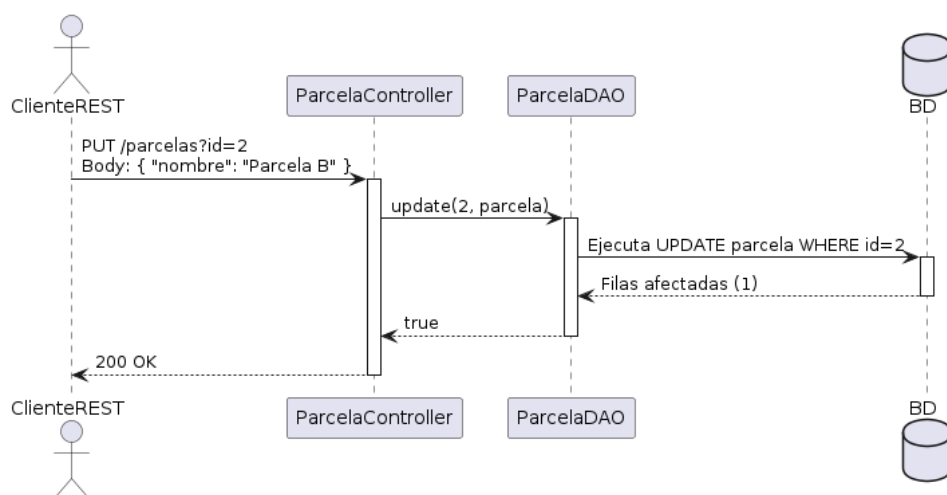


Figura 2: Operación PUT de Parcelas

4.2.2. Operación POST de Medidas (SenML)

A continuación se detalla el proceso mediante el cual un nodo sumidero recibe las métricas de un dispositivo y las almacena. Se hace evidente el proceso de validación SenML, el cual no se ve reflejado en diagramas más simples.

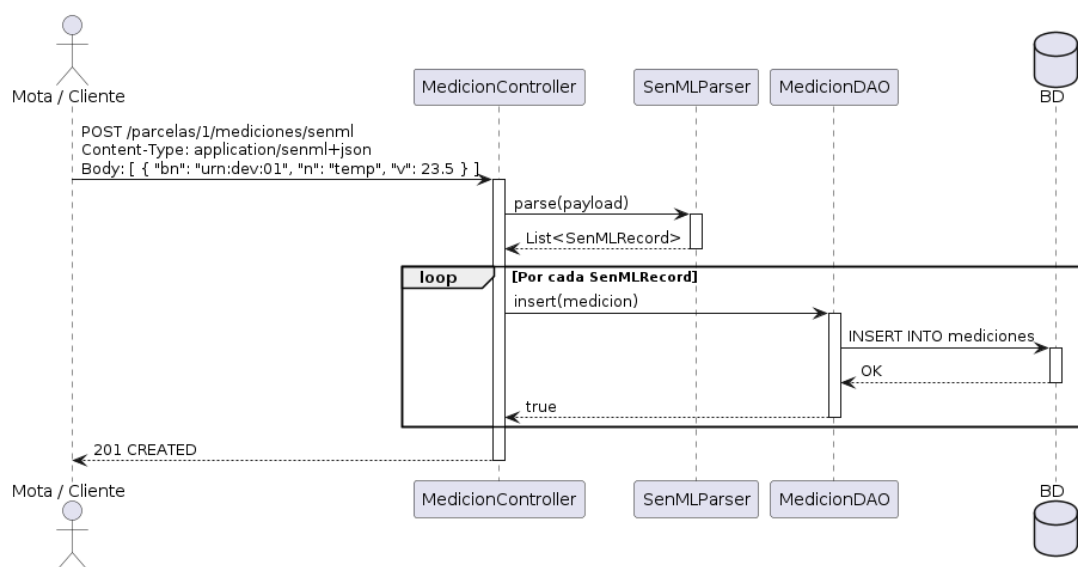


Figura 3: Operación POST de Medidas (SenML)

4.2.3. Operación DELETE de un Dispositivo

Este flujo muestra cómo se elimina una mota específica del sistema, garantizando que el controlador devuelve un *status code* 404 en caso de que el dispositivo no exista en la base de datos.

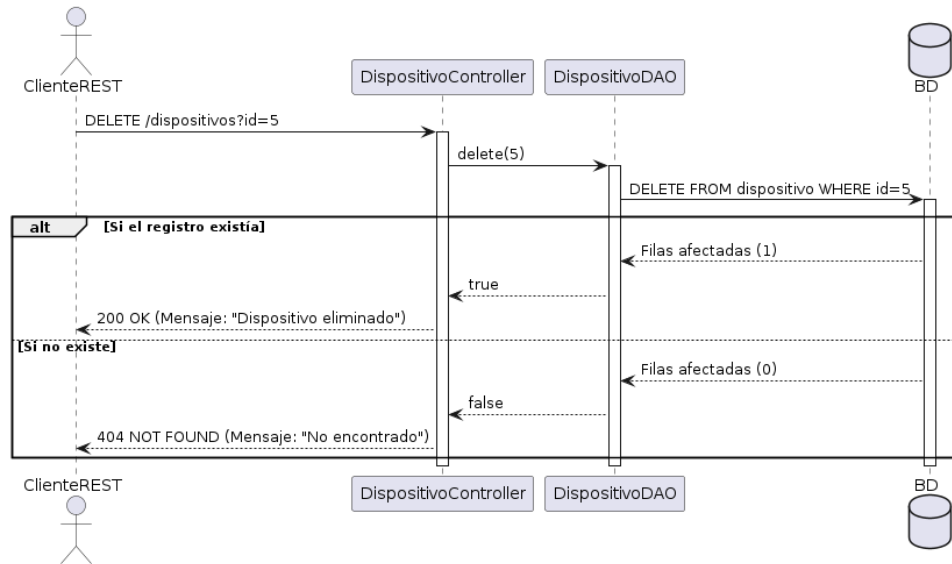


Figura 4: Operación DELETE de un Dispositivo

5. Conclusiones

El desarrollo de esta API RESTful ha demostrado la eficacia de utilizar tecnologías estandarizadas, como Spring Boot y el RFC 8428 (SenML), para gestionar la adquisición de datos masivos en redes de IoT. La aplicación paulatina de los conceptos teóricos de la asignatura, desde los mappings básicos en GET hasta la definición exhaustiva de un CRUD utilizando códigos de estado semánticos y la inyección de parámetros mediante `RequestParam` y `PathVariable`, garantizan que el sistema no solo funcione, sino que cumpla con los estándares de calidad de la industria del software.

Por último, los patrones de diseño aplicados facilitan la extensibilidad del código, de modo que en el futuro la sustitución del almacenamiento en memoria por una base de datos robusta, o la integración con interfaces gráficas avanzadas, podrán realizarse sin impactar la capa de red del sistema.